ASTRA User Manual

Version: 8.0 | **Date:** June 27, 2026
Authors: Glenn J. White, Open University and Rutherford Appleton Laboratory, England

Repository: https://github.com/Tilanthi/ASTRA

Table of Contents

1. Introduction
2. System Architecture
3. Installation and Setup
4. Getting Started
5. New in Version 8.0
6. Core Capabilities Overview
7. V4.0 Revolutionary Capabilities
8. V5.0 Discovery Enhancement System
9. V7.0 Autonomous Research Scientist
10. Automatic Startup Discovery
11. Use Case Examples
12. Advanced Features
13. Domain Modules
14. API Reference
15. Best Practices
16. Troubleshooting
17. Performance Optimization
18. Development Workflow
19. Testing and Verification
20. Appendices

1. [■ New in Version 9.0](#5-new-in-version-90)

2. [Core Capabilities Overview](#6-core-capabilities-overview)

3. [V4.0 Revolutionary Capabilities](#7-v40-revolutionary-capabilities)

4. [V5.0 Discovery Enhancement System](#8-v50-discovery-enhancement-system)

5. [V7.0 Autonomous Research Scientist](#9-v70-autonomous-research-scientist)

6. [■ Automatic Startup Discovery](#10-automatic-startup-discovery)

7. [Use Case Examples](#11-use-case-examples)

8. [Advanced Features](#12-advanced-features)

9. [Domain Modules](#13-domain-modules)

10. [API Reference](#14-api-reference)

11. [Best Practices](#15-best-practices)

12. [Troubleshooting](#16-troubleshooting)

13. [Performance Optimization](#17-performance-optimization)

14. [Development Workflow](#18-development-workflow)

15. [Testing and Verification](#19-testing-and-verification)

16. [Appendices](#20-appendices)

1. Introduction

1.1 What is ASTRA?

ASTRA (Autonomous Scientific Discovery in Astrophysics) is an integrated computational framework that combines numerical data analysis, causal reasoning, physical validation, and statistical inference to enable automated scientific discovery in astrophysics. Unlike traditional machine learning systems that detect patterns without understanding their physical meaning, or large language models that can explain concepts but cannot process numerical data, ASTRA integrates multiple analytical approaches to provide physically interpretable, validated scientific insights.

Version 9.0 represents the production-hardened evolution of the system:

• **Fiction-free discovery pipeline (v14.x)** - Every candidate passes a machine-checkable two-gate EVALUATE on real astronomical data before promotion; fictional or mock discoveries are impossible by construction

• **Evolutionary discovery (AlphaEvolve-inspired)** - Program-space search scored by executable evaluation on real archives

• **Token economy** - A daily token-budget circuit breaker keeps autonomous operation within configurable cost limits

• **Paper tooling** - A de-AI text audit battery and a reference-verification gate for bibliographies

• **Always-on service** - A macOS LaunchAgent supervisor with auto-restart and user-yielding behaviour

1.2 Key Design Principles

Physics-Aware Reasoning: All discoveries are validated against fundamental physical principles including conservation laws, dimensional consistency, and established theoretical frameworks.

Causal Understanding: ASTRA distinguishes between correlation and causation using structural causal models, enabling identification of physical mechanisms rather than mere associations.

Uncertainty Quantification: Every result includes properly propagated uncertainties, confidence intervals, and statistical significance assessments.

Reproducibility: All analyses are fully documented and reproducible, with complete provenance tracking from raw data to final conclusions.

Autonomous Operation: V9.0 conducts continuous scientific discovery automatically in the background, 24/7.

Anti-Hallucination Protection: Persistent memory system prevents propagation of incorrect information.

1.3 Who Should Use This Manual?

This manual is written for expert users including:

• Research astronomers and astrophysicists

• Data scientists working with astronomical data

• Computational scientists requiring physics-aware analysis tools

• Graduate students and postdoctoral researchers in astrophysics

Users should have familiarity with:

• Python programming

• Basic statistical concepts

• Fundamental astrophysical principles

• Command-line operation

1.4 What's New in Version 9.0

Fiction-Free Discovery Pipeline (v14.x)

• **Two-gate EVALUATE** - Candidates must pass machine-checkable evaluation on real data, twice

• **Niche rotation, deduplication, and self-improvement** over a curated non-optical + optical data lake

• **Anti-circularity gate** - Prevents a discovery from being validated against data it was derived from

• **Promotion verification** - Store promotions are independently re-verified at artifact level

Evolutionary Discovery

• **AlphaEvolve-style program search** with machine-checkable fitness on real archives

• **Four-level discovery classification** (Novel Observation / Theoretical Insight / Paradigm Shift / Eureka) and 3-D scoring (Novelty, Validation, Impact)

Token Economy & Engineering Rules (v14.20 / v14.21)

• **Daily token budget** (`ASTRA_DAILY_TOKEN_BUDGET`, default 2M tokens/day) checked before every dispatch

• **Engineering-rules package** encoding the standing correctness rules as tests

Paper Tooling

• **DEAI battery** (`astra_core/deai/`) - audit of drafts for AI-typical text patterns

• **Reference verification gate** (`astra_core/reference_verification.py`) - DOI/arXiv round-trip verification of bibliographies; fabricated or misattributed references fail the gate

Operation

• **`com.astra.discovery` LaunchAgent** - starts at login, auto-restarts on crash, yields to user activity (the legacy `start_continuous_discovery.sh` watchdog is deprecated)

See [Section 5](#5-new-in-version-90) for detail.

2. System Architecture

2.1 Architectural Overview

ASTRA implements a layered architecture designed for astrophysical data analysis and inference:

```
■■■
```

• State persistence across sessions

• Resource-aware throttling

• Four operation modes: CONTINUOUS, INTELLIGENT (default), IDLE, OFF

Discovery State Tracking:

• STARTING → RUNNING → PAUSED/THROTTLED → STOPPING → STOPPED

• Cycle counter and discovery history

• Last discovery timestamp

• Resource usage monitoring

2.2.2 PHOTON-Inspired Hierarchical Processing (NEW)

Multi-Resolution Context Representation (`astra_core/reasoning/hierarchical_context_processor.py`)

• 4-level hierarchical compression (FINE → MID → COARSE → ABSTRACT)

• 5-10 x performance improvement for complex queries

• 10 x memory reduction for long contexts

• 64 x compression ratio at top level

Chunk-Local Parallel Reasoning (`astra_core/reasoning/chunk_local_parallel.py`)

• Automatic chunk identification and dependency analysis

• Parallel processing of independent chunks

• 4-8 x improvement for hypothesis generation

• Near-linear scaling up to CPU core count

Hierarchical Knowledge Compression (`astra_core/knowledge/hierarchical_knowledge_compressor.py`)

• Bottom-up compression (Observations → Parameters → Principles → Theories)

• Top-down reconstruction (Theories → Principles → Parameters → Predictions)

• 3-5 x improvement for knowledge-intensive tasks

• 100 x compression for large datasets

2.2.3 V4.0 Revolutionary Capabilities

Meta-Context Engine (MCE)

• Multi-layered context representation across 7 dimensions

• Temporal, perceptual, domain, modality, epistemic, social, emotional layers

• Context layering and blending

• Dynamic context updates

Autocatalytic Self-Compiler (ASC)

• Self-improving system architecture

• Version management and compilation

• Metaprogramming capabilities

• Recursive self-enhancement

Cognitive-Relativity Navigator (CRN)

• Adaptive abstraction navigation (0-100 scale)

• Multi-level reasoning control

• Concept climbing and descending

• Abstraction scale management

Multi-Mind Orchestration (MMOL)

• 7 specialized minds: Physics, Empathy, Politics, Poetry, Mathematics, Causal, Creative

• Parallel reasoning with mind arbitration

• Anticipatory confidence prediction

• Conflict resolution

2.2.4 Physics Engine

Location: `astra_core/physics/`

The Physics Engine implements fundamental physical laws and constraints:

• UnifiedPhysicsEngine with 8+ models

• Relativistic Physics (black holes, cosmology)

• Quantum Mechanics (atomic processes, spectroscopy)

• Nuclear Astrophysics (nucleosynthesis, stellar evolution)

• Differentiable Physics (gradient-based optimization)

• Physics Curriculum Learning (15 stages)

Key Constants (CGS units):

• G = 6.674e-8 (gravitational)

• c = 2.998e10 (speed of light)

• h = 6.626e-27 (Planck)

• k_B = 1.381e-16 (Boltzmann)

• M_sun = 1.989e33 (solar mass)

2.2.5 Causal Reasoning Module

Location: `astra_core/causal_discovery/`

Advanced causal discovery capabilities:

• Structural Causal Models

• PC Algorithm implementation

• Temporal Causal Discovery

• Counterfactual Analysis

• Causal Graph Visualization

2.2.6 Memory & Anti-Hallucination Systems

Persistent Memory System (`astra_core/memory/persistent/`)

• Session initialization and state restoration

• Hallucination register (known incorrect claims)

• Claim verification before output

• Session checkpoints

• Knowledge persistence across sessions

Memory Hierarchies:

• MORK Ontology (concept hierarchies)

• Memory Graph (context relationships)

• Working Memory (7+/-2 capacity constraint)

• Episodic Memory (event sequences)

• Semantic Memory (facts and concepts)

3. Installation and Setup

3.1 System Requirements

Minimum Requirements:

• Python 3.8 or higher

• 8 GB RAM

• 2 GB free disk space

• Linux, macOS, or Windows with WSL2

Recommended for Large Datasets:

• Python 3.10 or higher

• 32 GB RAM

• 20 GB free disk space

• SSD storage for better I/O performance

• Multi-core processor (4+ cores)

Optional Dependencies:

• GPU for accelerated deep learning (CUDA-capable)

• Jupyter for interactive analysis

• Docker for containerized deployment

3.2 Installation Methods

3.2.1 Installation from GitHub

```
# Clone the repository git clone https://github.com/Tilanthi/ASTRA.git cd ASTRA # Install in
editable mode pip install -e . # Install optional dependencies pip install -e ".[dev]"
```

3.2.2 Verification of Installation

```
# Test installation python -c "import astra_core; print('ASTRA installed successfully')" #
Check version python -c "from astra_core import version; print(version)" # Run comprehensive
system test python astra_core/comprehensive_system_test.py # Expected: All 18+ capabilities
PASS (100%)
```

3.3 Configuration

3.3.1 Basic Configuration

Create a configuration file `~/.astra/config.json`:

```
{ "data_directory": "~/astronomy_data", "memory_limit_gb": 16, "num_workers": 4, "log_level":
"INFO" }
```

3.3.2 ■ Discovery Configuration (NEW)

```
from astra_core.autonomous_startup_discovery import ( StartupDiscoveryConfig,
StartupDiscoveryMode ) # Default configuration (used automatically) config =
StartupDiscoveryConfig( mode=StartupDiscoveryMode.INTELLIGENT, startup_delay_seconds=5,
idle_threshold_seconds=300, discovery_interval_seconds=1800,
enable_literature_monitoring=True, enable_hypothesis_generation=True,
enable_data_analysis=True, enable_theoretical_discovery=True, enable_causal_discovery=True,
primary_domains=['astrophysics', 'astronomy', 'cosmology'], report_discoveries=True )
```

3.4 Running ASTRA from Claude Code

Claude Code is Anthropic's official CLI for Claude, providing direct integration with ASTRA for autonomous scientific research.

3.4.1 Installation

```
# Install Claude Code via npm npm install -g @anthropic/claude-code # Or via Homebrew (macOS)
brew install claude-code # Verify installation claude-code --version
```

3.4.2 Basic Usage

Interactive Mode:

```
# Start Claude Code with ASTRA claude-code --astra # Or navigate to ASTRA directory and start
cd /Users/gjw255/astrodata/SWARM/ASTRA-dev-main claude-code
```

Direct Commands:

```
# Ask ASTRA a question (discovery automatically pauses during query) claude-code "Using ASTRA,
explain why filament widths cluster at 0.1 pc" # Run autonomous research (discovery continues
in background) claude-code "Use ASTRA's V7.0 autonomous scientist to study interstellar
filaments"
```

4. Getting Started

4.1 Your First Analysis

Example 1: Basic Query with ■ Automatic Discovery

```
from astra_core import create_stan_system # Create system - ■ Discovery starts AUTOMATICALLY!
system = create_stan_system() # Check discovery status status = system.get_discovery_status()
print(f"Discovery state: {status['state']}") # Output: Discovery state: running # Ask a
question - discovery automatically pauses result = system.answer("What is the typical
temperature of a molecular cloud?") print(result['answer']) # Output: "Molecular clouds
typically have temperatures in the range 10-20 K..." # Discovery automatically resumes after
query completes status = system.get_discovery_status() print(f"Discovery state:
{status['state']}") # Output: Discovery state: running
```

Example 2: Data Analysis with Hierarchical Processing

```
from astra_core import create_stan_system system = create_stan_system() # Load data data =
system.load_data("my_catalog.csv") # Analyze scaling relation (uses PHOTON hierarchical
processing) result = system.discover_scaling_relation( data, x_variable="luminosity",
y_variable="mass", question="How does luminosity scale with mass?" ) # Output: L proportional
to M^(3.5+/-0.2) with hierarchical compression
```

Example 3: ■ Manual Discovery Control

```
from astra_core import create_stan_system from astra_core.autonomous_startup_discovery import
get_autonomous_startup_discovery # Create system system = create_stan_system() # Get discovery
instance discovery = get_autonomous_startup_discovery() # Manual pause for critical task
discovery.pause("Running critical analysis") # ... perform critical task ... # Resume discovery
discovery.resume() # Check detailed status status = discovery.get_status() print(f"State:
{status['state']}") print(f"Cycles completed: {status['cycles_completed']}")
print(f"Discoveries made: {status['discoveries_made']}")
```

4.2 Understanding ASTRA's Output

ASTRA provides structured output:

Results: Primary answer with precision and units **Confidence**: Statistical confidence intervals **Methodology**: Methods used and parameters **Validation**: Physical constraint checks **Provenance**: Complete processing record **Recommendations**: Suggestions for further analysis ■ **Discovery Status**: Current autonomous discovery state

5. ■ New in Version 9.0

5.1 Fiction-Free Discovery Pipeline (v14.x)

The discovery path is built so that a fictional result cannot reach a store. Every candidate proposal is evaluated by an **executable two-gate EVALUATE**: a claim becomes a discovery only if independent code, run on real archival data, reproduces it — first at candidate level, again at promotion. An **anti-circularity gate** blocks validation against the same data the claim was derived from, and store promotions are re-verified at artifact level (papers are treated as tested tools: each published claim is paired with the test that verifies it).

Supporting machinery: a curated data lake spanning optical and non-optical archives, niche rotation to avoid over-mining one dataset, near-duplicate detection against the existing discovery memory, a self-improvement loop over residual failures, and a cross-modal cross-match lane (`sdss_wise_xmatch`).

```
from astra_core import create_stan_system system = create_stan_system() # chokepoint-gated; no
path to mock data result = system.query("anomaly survey of quasar hosts")
```

5.2 Evolutionary Discovery (AlphaEvolve-inspired)

Program-space evolution with machine-checkable fitness on real data: candidate programs mutate, and only those whose evaluation improves on real archives survive. The evaluation lane is the same two-gate EVALUATE, so evolved programs are held to the same fiction-free standard.

5.3 Token Economy and Budget Circuit Breaker (v14.20)

Autonomous operation is bounded by `ASTRA_DAILY_TOKEN_BUDGET` (default 2M tokens/day), checked before every LLM dispatch. When the budget is exhausted, dispatch stops until the next window — the system

degrades to local analysis rather than overspending.

5.4 Reference Verification Gate

AI-assembled bibliographies are audited by `astra_core/reference_verification.py`: every cited entry is round-tripped through Crossref/arXiv, checking that the DOI resolves **to the paper meant** (title similarity + author overlap), that preprints are cited in published form, that placeholders are resolved, and that the cite/bib sets reconcile in both directions. Exit code 1 on any issue.

```
python3 astra_core/reference_verification.py
```

5.5 Always-On Service (`com.astra.discovery`)

The supported continuous path is the macOS LaunchAgent: it runs the fiction-free supervisor, starts at login, auto-restarts on crash, and yields to user activity.

```
launchctl list com.astra.discovery # status tail -f .astra_service.log # logs launchctl
kickstart -k gui/$(id -u)/com.astra.discovery # restart
```

The legacy `start_continuous_discovery.sh` watchdog is **deprecated**.

5.6 Updated Statistics

Current System Statistics (V9.0):

• **Source files**: ~1,750 (astra_core, pycache-free)

• **Test suites**: `tests/test_all.py` (core), `test_token_economy.py`, `test_engineering_rules.py`, `test_discovery_chokepoint.py`, `test_reference_verification.py`

• **Pipeline gates**: citation resolution, derivation traces, tolerance checks, fiction-free EVALUATE, anti-circularity, promotion verification

• **Paper tooling**: DEAI battery (protocol v3.0), reference verification gate

• **Discovery stores**: genuine / evolved / memory, all chokepoint-gated

6. Core Capabilities Overview

ASTRA integrates 20+ analytical capabilities across multiple domains.

6.1 Causal and Statistical Analysis

6.1.1 Causal Discovery

```
result = system.perform_causal_inference( data=time_series, variables=["mass", "luminosity",
"star_formation_rate"], method="pc_algorithm" # or "ges", "lingsam", "fci" ) # Output: Causal
graph with edge directions and confidence scores
```

6.1.2 Temporal Causal Discovery

```
result = system.temporal_causal_discovery( data=time_series_data, time_lags=[1, 2, 3, 4, 5],
change_point_detection=True ) # Output: Time-lagged causal relationships with regime changes
```

6.1.3 Counterfactual Analysis

```
result = system.counterfactual_analysis( data=observational_data,
intervention="increase_temperature_by_10K", outcome="filament_width" ) # Output: Predicted
effect of intervention on outcome
```

6.1.4 Scaling Relations Discovery

```
result = system.discover_scaling_relation( data=cluster_data, x_variable="temperature",
y_variable="luminosity" ) # Output: L proportional to T^(2.5+/-0.3) with confidence intervals
```

6.2 Data Integration and Analysis

6.2.1 Multi-Wavelength Fusion

```
result = system.fuse_multiwavelength( catalogs={ "xray": xmm_catalog, "optical": hst_catalog,
"ir": spitzer_catalog }, matching_radius=2.0 # arcsec ) # Output: Unified catalog with
cross-matched sources
```

6.2.2 Bias Detection

```
result = system.detect_bias( data=galaxy_catalog, bias_type="malmquist", # or "selection",
"observational" flux_column="apparent_magnitude" ) # Output: Quantified bias and corrected
measurements
```

6.3 Advanced Reasoning

6.3.1 Swarm Reasoning

```
result = system.swarm_reasoning( question="What causes filament width variations?",
num_agents=10, iterations=5 ) # Output: Consensus answer with confidence distribution
```

6.3.2 Hierarchical Bayesian Meta-Learning

```
result = system.hierarchical_bayesian_learning( data=multi_dataset, model_type="power_law",
hierarchy_structure=["cluster", "galaxy", "region"] ) # Output: Posterior distributions with
hierarchical structure
```

7. V4.0 Revolutionary Capabilities

7.1 Meta-Context Engine (MCE)

The Meta-Context Engine provides multi-layered context representation across 7 dimensions.

Creating MCE:

```
from astra_core.metacognitive.meta_context_engine import create_meta_context_engine mce =
create_meta_context_engine() # Layer context across dimensions result = mce.layer_context(
query="Explain filament formation", dimensions=["temporal", "perceptual", "domain", "modality",
"epistemic"] ) # Output: Multi-layered context representation
```

Context Dimensions:

• **Temporal**: Past, present, future perspectives

• **Perceptual**: Different observation angles

• **Domain**: ISM, star formation, turbulence

• **Modality**: Text, numerical, visual

• **Epistemic**: Knowledge, belief, certainty levels

• **Social**: Collaborative perspectives

• **Emotional**: Impact and significance

7.2 Autocatalytic Self-Compiler (ASC)

The ASC enables self-improvement and metaprogramming.

Creating ASC:

```
from astra_core.metacognitive.autocatalytic_self_compiler import create_self_compiler asc =
create_self_compiler() # Compile and improve system result = asc.compile_system(
target="optimize_filament_analysis", optimization_level="aggressive" ) # Output: Improved
system with version tracking
```

Self-Improvement Features:

• Automatic performance optimization

• Bug detection and fixing

• Algorithm refinement

• Memory optimization

7.3 Cognitive-Relativity Navigator (CRN)

The CRN enables adaptive abstraction navigation across scales.

Creating CRN:

```
from astra_core.metacognitive.cognitive_relativity_navigator import create_navigator crn =
create_navigator() # Navigate abstraction scales result = crn.navigate_abstraction(
query="filament_structure", target_scale=50, # 0=atomic, 50=conceptual, 100=philosophical
direction="climb" # or "descend" ) # Output: Concept at target abstraction level
```

Abstraction Scale:

• 0: Atomic facts

• 25: Concrete mechanisms

• 50: Concepts and principles

• 75: Theoretical frameworks

• 100: Pure philosophy

7.4 Multi-Mind Orchestration (MMOL)

The MMOL coordinates 7 specialized minds for comprehensive analysis.

Creating MMOL:

```
from astra_core.metacognitive.multi_mind_orchestration import create_mind_orchestrator mmol =
create_mind_orchestrator() # Query with specific mind result = mmol.query_with_mind(
question="Explain filament formation", mind="physics" # or "empathy", "politics", "poetry",
"mathematics", "causal", "creative" ) # Output: Analysis from perspective of specified mind
```

The 7 Minds:

1. **Physics**: Physical laws and mechanisms

2. **Empathy**: Human impact and relevance

3. **Politics**: Policy implications

4. **Poetry**: Beauty and wonder

5. **Mathematics**: Mathematical structure

6. **Causal**: Cause and effect relationships

7. **Creative**: Novel hypotheses and ideas

8. V5.0 Discovery Enhancement System

8.1 Overview

V5.0 adds advanced discovery capabilities for genuine scientific discovery.

8.2 Capabilities

8.2.1 Genuine Discovery Detection

```
# Discover novel relationships result = system.genuine_discovery( data=survey_data,
knowledge_base="astrophysics_ontology", novelty_threshold=0.95 ) # Output: Novel discoveries
with significance scores
```

8.2.2 Physical Model Discovery

```
# Discover underlying physical models result = system.discover_physical_model(
data=observations, model_space=["power_law", "exponential", "broken_power_law"] ) # Output:
Best-fitting model with parameters
```

8.2.3 Adversarial Hypothesis Framework

```
# Devil's advocate reasoning result = system.adversarial_evaluation( hypothesis="Filament width
set by sonic scale", alternative_hypotheses=["magnetic_tension", "thermal_pressure"],
evidence_data=filament_observations ) # Output: Hypothesis comparison with Bayes factors
```

9. V7.0 Autonomous Research Scientist

9.1 Overview

The V7.0 Autonomous Research Scientist conducts the entire scientific research cycle:

```
Question → Hypothesis → Experiment → Analysis → Theory → Publication
```

9.2 Core Components

9.2.1 Question Generator

```
from astra_core.v7_autonomous_research import create_v7_scientist scientist =
create_v7_scientist() # Generate research questions questions =
scientist.generate_research_questions( domain="interstellar_medium", context={"focus":
"filament_widths"}, num_questions=5 ) # Example output: # Question 1: "What determines the
characteristic width of # interstellar filaments across different density regimes?" #
Importance: CRITICAL # Novelty: HIGH
```

9.2.2 Hypothesis Formulator

```
# Formulate hypotheses hypotheses = scientist.formulate_hypotheses( question=questions[0],
hypothesis_types=[HypothesisType.THEORETICAL, HypothesisType.EMPIRICAL] ) # Example output: #
Hypothesis 1: "Filament width is set by the sonic scale of # turbulent cascade, where velocity
dispersion equals thermal # sound speed (lambda_sonic approximately 0.1 pc for typical
conditions)" # Testability: HIGH # Falsifiability: HIGH
```

9.2.3 Experiment Designer

```
# Design experiment experiment = scientist.design_experiment( hypothesis=hypotheses[0],
experiment_type=ExperimentType.OBSERVATIONAL_TEST, constraints={"telescope": "ALMA", "time":
"10_hours"} ) # Example output: # Experiment Design: # - Target: 5 molecular clouds spanning
density range 10^2-10^5 cm^-3 # - Observations: High-resolution N2H+ mapping # - Measurements:
Filament widths, velocity dispersions, temperatures # - Required sensitivity: sigma_N
approximately 10^11 cm^-2
```

9.2.4 Publication Engine

```
# Generate publication paper = scientist.generate_publication( research_cycle={ "question":
questions[0], "hypotheses": hypotheses, "experiment": experiment, "results": results,
"analysis": analysis, "theory": theory }, format="A&A;", # or "MNRAS", "ApJ"
include_figures=True ) # Output includes: # - Complete paper text # - Publication-quality
figures # - Tables with measurements
```

10. ■ Automatic Startup Discovery

10.1 Overview

Version 8.0 introduces **Automatic Startup Discovery**, a transformative feature that enables continuous autonomous scientific discovery without manual activation.

10.2 How It Works

Startup Flow:

```
1. User calls: system = create_stan_system() 2. System initializes domains, physics, etc. 3.
_initialize_autonomous_discovery() is called 4. AutonomousStartupDiscovery instance created 5.
Discovery thread started automatically 6. Discovery runs in background
```

Query Processing Flow:

```
1. User calls: system.answer("query") 2. _handle_user_task_start() called 3. Discovery
pauses/throttles 4. Query processes normally 5. _handle_user_task_complete() called 6.
Discovery resumes 7. Result returned to user
```

10.3 Usage Examples

Basic Usage (Automatic)

```
from astra_core import create_stan_system # Discovery starts automatically! system =
create_stan_system() # Check status status = system.get_discovery_status() print(f"Discovery:
{status['state']}") # Queries work normally with automatic pause/resume result =
system.answer("What is star formation?")
```

Manual Control (Advanced)

```
from astra_core.autonomous_startup_discovery import get_autonomous_startup_discovery discovery
= get_autonomous_startup_discovery() # Manual pause discovery.pause("Manual maintenance") #
Resume discovery.resume() # Check detailed status status = discovery.get_status()
print(f"State: {status['state']}") print(f"Cycles completed: {status['cycles_completed']}")
```

Integration with Queries

```
# answer() handles everything automatically for query in user_queries: result =
system.answer(query) # Discovery automatically pauses during query # Discovery automatically
resumes after query
```

10.4 Discovery Modes

```
from astra_core.autonomous_startup_discovery import StartupDiscoveryMode # CONTINUOUS: Always
running, minimal throttling # INTELLIGENT (default): Adapts based on user activity # IDLE: Only
runs when user inactive (5+ minutes) # OFF: Discovery disabled
```

10.5 State Persistence

Discovery state is automatically saved to `~/.astra_persistent/startup_discovery_state.json` and includes:

• Discovery history

• Cycle count

• Last discoveries

• Resource usage metrics

11. Use Case Examples

11.1 Interstellar Medium Analysis

Question: "Why do filament widths cluster at 0.1 pc?"

```
from astra_core import create_stan_system system = create_stan_system() # Load Herschel data
herschel_data = system.load_data("herschel_filaments.fits") # Analyze filament widths result =
system.analyze_filaments( data=herschel_data, analysis_type="width_distribution",
compare_regions=["Aquila", "Polaris", "Taurus"] ) # Output: Width = 0.103 +/- 0.008 pc,
independent of density
```

11.2 Exoplanet Detection

```
# Detect exoplanets using radial velocity result = system.detect_exoplanets(
rv_data=hipparchos_timeseries, method="bayesian_periodogram", min_planets=1, max_planets=5 )
```

11.3 Galaxy Evolution

```
# Study galaxy scaling relations result = system.discover_scaling_relation(
data=galaxy_catalog, x="stellar_mass", y="star_formation_rate", control_variables=["redshift",
"environment"] )
```

12. Advanced Features

12.1 Multi-Mind Orchestration

```
# Query with specific mind result = system.answer_with_mind( question="Explain filament
formation", mind="physics" # or other minds )
```

12.2 Global Coherence Layer

```
# Ensure global coherence across analyses result = system.coherent_analysis( questions=[ "What
causes filament width variations?", "How does magnetic field affect filaments?", "What is the
role of turbulence?" ], coherence_threshold=0.8 )
```

12.3 Hierarchical Processing

```
# Use PHOTON-inspired hierarchical processing result = system.process_query_hierarchical(
query="Complex multi-domain question", enable_compression=True, compression_levels=4 )
```

13. Domain Modules

ASTRA includes **75 specialized domain modules** organized into categories:

12.4 DEAI Text Audit Battery

`astra_core/deai/deai_battery_v3.py ` audits changed text for AI-typical patterns (emphasis habits, connective rates, edit budgets) and reports per-check pass/fix status. Run near the end of any writing task, on request.

12.5 Paper Gates and the Referee Loop

Recurring paper work uses gate specs (`roundN_gates.json`) + the standing runner (`tools/virtual_referee/gates.py`), quote sweeps, and a state report per round. Bibliographies additionally pass the reference verification gate (Section 5.4).

13.1 Domain Categories

Stellar Astrophysics (8 domains): stellar_structure, stellar_atmospheres, stellar_populations, nuclear_astrophysics, compact_binaries, xray_binaries, supernovae, exoplanet_atmospheres

Interstellar Medium & Star Formation (8 domains): ism, molecular_cloud_dynamics, molecular_cloud_evolution, molecular_cloud_collapse, star_formation, hii_regions, dust_formation, dust_grain_physics

Exoplanets & Solar System (4 domains): exoplanets, planetary_formation, solar_system, orbital_dynamics

High-Energy Astrophysics (5 domains): high_energy, agn, gamma_ray, astroparticle, gravitational_waves

Galaxy Evolution & Structure (8 domains): galaxy_evolution, galaxy_clusters, dwarf_galaxies, galactic_structure, galactic_archaeology, extragalactic, intergalactic_medium, large_scale_structure

Compact Objects & Extreme Physics (7 domains): black_holes, accretion_disk_theory, tidal_disruption, kilonovae, general_relativity, gravitational_lensing, frbs

Observational Techniques & Wavelengths (12 domains): radio_galactic, radio_extragalactic, millimetre_astronomy, submillimeter_astronomy, infrared_astronomy, farinfrared_astronomy, interferometry, polarimetry, astrometry, time_domain, cmb, xray_binaries

Theoretical & Computational Physics (10 domains): theoretical_astrophysics, computational_astrophysics, numerical_methods, mhd, plasma_physics, fluid_dynamics, statistical_mechanics, quantum_applications, solid_state_astro, dynamical_systems

Radiation & Atomic Physics (6 domains): radiative_processes, radiative_transfer_theory, photoionization, atomic_physics, molecular_spectroscopy, astrochemical_surveys

Solar & Heliospheric Physics (2 domains): solar_physics, heliospheric_physics

Specialized & Cross-Disciplinary (5 domains): cosmology, prebiotic_chemistry, signal_processing, inverse_problems, hpc

13.2 Using Domain Modules

```
# Load specific domain from astra_core.domains import load_domain ism_domain =
load_domain("ism") # Query domain result = ism_domain.process_query( query="Calculate Jeans
length in molecular cloud", context={"density": 1e4, "temperature": 10} )
```

14. API Reference

14.1 Main System API

```
class UnifiedSTANSystem: """Main ASTRA system""" def answer(self, question: str) -> Dict:
"""Answer a natural language question with automatic pause/resume""" def process_query(self,
query: str, context: Dict = None) -> Dict: """Process a query with optional context""" def
get_discovery_status(self) -> Dict: """Get current autonomous discovery status""" def
discover_scaling_relation(self, data, x, y) -> Dict: """Discover scaling relation""" def
perform_causal_inference(self, data, variables) -> Dict: """Perform causal discovery""" def
detect_bias(self, data, bias_type) -> Dict: """Detect observational biases"""
```

14.2 ■ Autonomous Startup Discovery API

```
class AutonomousStartupDiscovery: """Autonomous discovery system""" def get_status(self) ->
Dict: """Get detailed discovery status""" def pause(self, reason: str): """Manually pause
discovery""" def resume(self): """Resume paused discovery""" def stop(self): """Stop discovery
completely""" def get_discoveries(self) -> List: """Get list of discoveries made"""
```

14.3 V4.0 Capabilities API

```
# Meta-Context Engine def create_meta_context_engine() -> MetaContextEngine: """Create MCE
instance""" def layer_context(query, dimensions) -> Dict: """Layer context across dimensions"""
# Autocatalytic Self-Compiler def create_self_compiler() -> AutocatalyticSelfCompiler:
"""Create ASC instance""" # Cognitive-Relativity Navigator def create_navigator() ->
CognitiveRelativityNavigator: """Create CRN instance""" # Multi-Mind Orchestration def
create_mind_orchestrator() -> MindOrchestrator: """Create MMOL instance"""
```

15. Best Practices

15.1 Data Preparation

• Use standard formats (FITS, CSV, HDF5)

• Include proper metadata

• Propagate uncertainties

• Document data provenance

15.2 Query Formulation

• Be specific about your question

• Provide context when relevant

• Specify desired output format

• Include constraints if applicable

15.3 Result Interpretation

• Always check confidence intervals

• Validate against physical expectations

• Consider alternative explanations

• Reproduce analyses independently

15.4 ■ Discovery Management

• Let discovery run automatically for continuous insights

• Use `get_discovery_status()` to monitor progress

• Manually pause only for critical tasks

• Review discoveries regularly

15.5 Anti-Hallucination

• Always initialize persistent memory at session start

• Verify claims before outputting numerical values

• Update hallucination register when errors are found

• Check `~/.astra_persistent/hallucination_register.json`

16. Troubleshooting

16.1 Common Issues

Issue: System returns low confidence results

Solution:

• Provide more context

• Increase data quality

• Check for missing variables

• Consider alternative formulations

Issue: Memory errors with large datasets

Solution:

• Increase memory limit

• Use batch processing

• Reduce data resolution

• Enable memory-efficient algorithms

Issue: ■ Discovery not starting

Solution:

• Check logs for initialization errors

• Verify `system.get_discovery_status()['state']`

• Check `~/.astra_persistent/` directory permissions

• Ensure system properly initialized

Issue: Discovery interfering with queries

Solution:

• Manually pause during critical tasks

• Use `get_autonomous_startup_discovery().pause("reason")`

• Switch to IDLE mode for less aggressive discovery

Issue: Hallucinated values appearing

Solution:

• Initialize persistent memory at session start

• Use `verify_claim_before_output()` for numerical claims

• Update hallucination register

• Check `~/.astra_persistent/hallucination_register.json`

16.2 Getting Help

• Check GitHub issues

• Consult documentation

• Use verbose logging

• Run comprehensive system test

• Contact developers

17. Performance Optimization

17.1 Hierarchical Processing

For complex queries, enable PHOTON-inspired hierarchical processing:

```
# Enable hierarchical compression result = system.process_query_hierarchical( query="complex
multi-domain question", enable_compression=True, compression_levels=4, chunk_parallel=True ) #
Expected performance improvements: # - 5-10 x faster for complex queries # - 10 x memory
reduction for long contexts # - 4-8 x faster for hypothesis generation
```

17.2 Memory Management

```
# Configure memory limits from astra_core.autonomous_startup_discovery import
StartupDiscoveryConfig config = StartupDiscoveryConfig( mode=StartupDiscoveryMode.IDLE, # Less
resource intensive discovery_interval_seconds=3600 # Longer intervals )
```

17.3 Parallel Processing

```
# Use chunk-local parallel processing from astra_core.reasoning.chunk_local_parallel import
ChunkLocalParallelProcessor parallel_proc = ChunkLocalParallelProcessor( max_workers=8, #
Number of parallel workers chunk_size=1000 # Chunk size for parallelization )
```

18. Development Workflow

18.1 Post-Upgrade Verification Testing

CRITICAL: After any substantial upgrade to ASTRA functionality, comprehensive verification testing MUST be performed.

When to Run:

• Adding new domain modules

• Modifying core architecture

• Updating physics engine or models

• Changes to memory systems

• Adding or modifying reasoning capabilities

Test Procedure:

```
# Run comprehensive system test python astra_core/comprehensive_system_test.py # Expected
output: All 18 capabilities should PASS (100%)
```

Test Coverage:

• **75 Domain Modules**: Import, instantiation, and query handling

• **Memory Systems**: MORK Ontology, Context Graph, Working Memory

• **Physics Engine**: UnifiedPhysicsEngine with all models

• **Causal Discovery**: V50, V70, and astrophysical causal engines

• **Advanced Reasoning**: Swarm reasoning, hierarchical Bayesian

• **V4 Capabilities**: Meta-Context Engine, ASC, CRN, MMOL

• ■ **Autonomous Discovery**: Automatic startup and pause/resume

18.2 Code Organization

Capability Files:

• **V36-V94 capabilities**: `astra_core/capabilities/vXX_*.py`

• **Physics modules**: `astra_core/physics/*.py`

• **Domain modules**: `astra_core/domains//**init**.py`

• **Meta-learning**: `astra_core/reasoning/maml_optimizer.py`

Memory Hierarchy:

• **MORK Ontology**: `astra_core/memory/mork_ontology.py`

• **Memory Graph**: `astra_core/memory/context_graph.py`

• **Working Memory**: `astra_core/memory/working/`

• ■ **Persistent Memory**: `astra_core/memory/persistent/`

Test Files:

• **Comprehensive Test**: `astra_core/comprehensive_system_test.py`

• **Domain Validation**: `astra_core/tests/validation_benchmarks.py`

• **V4 Integration**: `astra_core/tests/v4/test_v4_integration.py`

18.3 Design Patterns

1. Capability Auto-Selection

```
# WRONG: Manual capability selection result = system.reasoning.causal_discovery(query) #
CORRECT: Let system auto-select result = system.answer(query) # Auto-selects best capabilities
```

2. Module Registration Pattern

```
from astra_core.domains import BaseDomainModule, register_domain @register_domain class
MyDomain(BaseDomainModule): def get_default_config(self): return DomainConfig(
domain_name="my_domain", version="1.0.0", keywords=["keyword1", "keyword2"],
capabilities=["capability1", "capability2"] )
```

3. Factory Function Pattern

```
# Use factory functions system = create_stan_system() mce = create_meta_context_engine()
optimizer = create_maml_optimizer(model_fn, loss_fn) # NOT: system = UnifiedSTANSystem() #
Avoid direct constructors
```

19. Testing and Verification

19.1 Running Tests

```
# Run V4.0 capability tests python astra_core/tests/v4/run_tests.py # Run specialist capability
tests (66 V45 capabilities) python astra_core/tests/test_specialist_capabilities.py # Run Phase
2-4 enhancement tests python astra_core/tests/test_phase_2_4.py # Run comprehensive system test
python astra_core/comprehensive_system_test.py
```

19.2 Test Individual Components

```
# Test physics modules python -c "from astra_core.physics.relativistic_physics import
RelativisticPhysics; print(RelativisticPhysics.schwarzschild_radius(1.989e33))" # Test domain
modules python -c "from astra_core.domains.high_energy import create_high_energy_domain; d =
```

```
create_high_energy_domain(); print(d.get_capabilities())" # Test MAML optimizer python -c "from
astra_core.reasoning.maml_optimizer import MAMLOptimizer; print('MAML imported')" # Test ■
autonomous startup python -c "from astra_core import create_stan_system; s =
create_stan_system(); print(s.get_discovery_status())"
```

19.3 Verification Report

After successful verification, document:

• Date and version of verification

• All 75 domains with PASS status

• All 18+ advanced capabilities with PASS status

• Cross-module dependency verification

• Any issues found and resolved

20. Appendices

Appendix A: Complete Capability List

1. Bias Detection

2. Scaling Relations Discovery

3. Causal Inference

4. Model Selection

5. Multi-Wavelength Fusion

6. Uncertainty Quantification

7. Temporal Analysis

8. Instrument-Aware Analysis

9. Anomaly Detection

10. Ensemble Prediction

11. Physical Model Discovery

12. Bayesian Model Selection

13. Counterfactual Analysis

14. Genuine Discovery Detection

15. **V4.0: Meta-Context Engine**

16. **V4.0: Autocatalytic Self-Compiler**

17. **V4.0: Cognitive-Relativity Navigator**

18. **V4.0: Multi-Mind Orchestration**

19. **V5.0: Adversarial Hypothesis Framework**

20. **V7.0: Question Generation**

21. **V7.0: Hypothesis Formulation**

22. **V7.0: Experiment Design**

23. **V7.0: Experiment Execution**

24. **V7.0: Theory Revision**

25. **V7.0: Publication Generation**

26. **V8.0: Automatic Startup Discovery**

27. **V8.0: Hierarchical Context Processing**

28. **V8.0: Chunk-Local Parallel Reasoning**

29. **V8.0: Hierarchical Knowledge Compression**

30. **V8.0: Recursive Generation**

31. **V8.0: Anti-Hallucination Protection**

32. ■ **V9.0: Fiction-free two-gate EVALUATE pipeline (v14.x)**

33. ■ **V9.0: Evolutionary (AlphaEvolve-style) discovery**

34. ■ **V9.0: Token economy + engineering-rules packages**

35. ■ **V9.0: Reference verification gate for bibliographies**

36. ■ **V9.0: DEAI text-audit battery**

Appendix B: Domain Module List

[Complete list of 75 domain modules organized by category - see Section 13.1]

Appendix C: Physical Constants

CGS Units:

• G = 6.674e-8 (gravitational constant)

• c = 2.998e10 (speed of light)

• h = 6.626e-27 (Planck constant)

• k_B = 1.381e-16 (Boltzmann constant)

• M_sun = 1.989e33 (solar mass)

• R_sun = 6.957e10 (solar radius)

• L_sun = 3.828e33 (solar luminosity)

• m_e = 9.109e-28 (electron mass)

• m_p = 1.673e-24 (proton mass)

• e = 4.803e-10 (elementary charge)

• *sigma* = 5.670e-5 (Stefan-Boltzmann constant)

Appendix D: Unit Conversions

Length:

• 1 pc = 3.086e18 cm

• 1 AU = 1.496e13 cm

• 1 R_sun = 6.957e10 cm

• 1 R_earth = 6.371e8 cm

Mass:

• 1 M_sun = 1.989e33 g

• 1 M_earth = 5.972e27 g

• 1 M_jup = 1.898e30 g

Energy:

• 1 eV = 1.602e-12 erg

• 1 keV = 1.602e-9 erg

• 1 MeV = 1.602e-6 erg

Temperature:

• 1 eV/k_B = 1.160e4 K

• 1 keV/k_B = 1.160e7 K

Index

A

• Abstraction scale, 100

• Anti-hallucination protection, 10, 48

• Autonomous research scientist, 63

• Autonomous startup discovery, 51, 66

C

• Causal discovery, 57

• Causal inference, 57

• Cognitive-relativity navigator, 61

• Configuration, 39

• Counterfactual analysis, 57

D

• Data analysis, 58

• Domain modules, 75

• Discovery enhancement, 62

F

• Factory functions, 84

• File organization, 83

H

• Hierarchical processing, 49

• Hierarchical context processor, 49

• Hypothesis formulation, 64

I

• Installation, 37

• Installation verification, 38

M

• Meta-context engine, 60

• Meta-learning, 60

• Multi-mind orchestration, 61

• Memory systems, 36

P

• Persistent memory, 48

• Physics engine, 36

• PHOTON implementation, 49

S

• Scaling relations, 58

• Self-compiler, 60

• Startup discovery, 51

• System architecture, 32

T

• Testing, 81

• Troubleshooting, 78

V

• V4.0 capabilities, 60

• V5.0 discovery, 62

• V7.0 autonomous scientist, 63

• V8.0 features, 45

• V9.0 fiction-free pipeline, 51

• V9.0 reference verification, 53

• V9.0 token economy, 54

Document Version: 8.0 **Last Updated**: June 27, 2026 **Authors**: Glenn J. White, Open University and Rutherford Appleton Laboratory, England **License**: [License information]

For the latest version, visit: https://github.com/Tilanthi/ASTRA

■ Quick Reference Card

Essential Commands

```
# Create system (automatic discovery) from astra_core import create_stan_system system =
create_stan_system() # Check discovery status status = system.get_discovery_status() # Answer
query (automatic pause/resume) result = system.answer("Your question") # Manual discovery
control from astra_core.autonomous_startup_discovery import get_autonomous_startup_discovery
discovery = get_autonomous_startup_discovery() discovery.pause("reason") discovery.resume() #
Initialize anti-hallucination protection from astra_core.memory.persistent import
create_integrator integrator = create_integrator() integrator.initialize_session() # Verify
claims result = integrator.verify_claim_before_output("claim")
```

File Locations

• **System**: `astra_core/`

• **Tests**: `astra_core/tests/`

• **Persistent State**: `~/.astra_persistent/`

• **Configuration**: `~/.astra/config.json`

• **Manual**: `User_Manual/User_Manual_V9.md`

Performance Tips

1. Enable hierarchical processing for complex queries

2. Use chunk-local parallel processing for independent tasks

3. Configure discovery mode based on workload

4. Initialize persistent memory at session start

5. Run comprehensive tests after upgrades

Common Patterns

```
# Factory pattern (correct) system = create_stan_system() # Auto-selection (correct) result =
system.answer(query) # Module registration (correct) @register_domain class
MyDomain(BaseDomainModule): pass
```

END OF DOCUMENT